

Rajalakshmi Engineering College

Name: Pavithra S
Email: 241001162@rajalakshmi.edu.in
Roll no: 241001162
Phone: 8122081287
Branch: REC
Department: IT - Section 2
Batch: 2028
Degree: B.E - IT

Scan to verify results



2024_28_III_OOPS Using Java Lab

REC_2028_OOPS using Java_Week 9_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Aarav is developing a music playlist application where users can manage their favorite songs. He wants to implement a feature that allows users to reorder the playlist by moving a song from one position to another.

You need to implement a function that performs the following operations using a LinkedList:

Add songs to the playlist in the given order. Move a song from a specified position to another position in the playlist. Print the final playlist after all operations.

Input Format

The first line of the input consists of an integer n representing the number of songs.

The next n lines, each containing a string representing a song name.

After the songs are given the next line contains an integer m, the number of move operations.

The next m lines, each containing two integers x and y representing the move operation where the song at position x (0-based index) should be moved to position y.

Output Format

The output prints the final playlist, each song on a new line.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5

SongA

SongB

SongC

SongD

SongE

2

2 4

0 3

Output: SongB

SongD

SongE

SongA

SongC

Answer

```
// You are using Java
```

```
import java.io.*;
```

```
import java.util.*;
```

```
class Main{
```

```
    public static void main(String[] args){
```

```
        Scanner sc=new Scanner(System.in);
```

```

int n=sc.nextInt();
sc.nextLine();

LinkedList<String> playlist=new LinkedList<>();

for(int i=0;i<n;i++){
    String song=sc.nextLine();
    playlist.add(song);
}

int m=sc.nextInt();

for(int i=0;i<m;i++){
    int x=sc.nextInt();
    int y=sc.nextInt();

    String s=playlist.remove(x);
    playlist.add(y, s);
}

for(int i=0;i<n;i++)
    System.out.println(playlist.get(i));
}
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

Sanjay is working on a program to merge two sorted linked lists into a single sorted list using Java's LinkedList class from the Collections framework. Given two sorted linked lists, he wants to merge them while maintaining the sorted order.

Write a Java program that:

Reads two sorted linked lists. Merges them into a single sorted linked list. Prints the merged list in ascending order.

Input Format

The first line contains an integer m (the size of the first linked list).

The second line contains m space-separated integers (sorted).

The third line contains an integer n (the size of the second linked list).

The fourth line contains n space-separated integers (sorted).

Output Format

The output prints the merged linked list as space-separated integers.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 2

5 10

3

1 3 8

Output: 1 3 5 8 10

Answer

```
import java.util.*;
class MergeSortedLinkedLists {
// You are using Java

    public static void main(String[] args) {
        Scanner sc=new Scanner(System.in);

        int m=sc.nextInt();
        LinkedList<Integer> l1=new LinkedList<>();

        for(int i=0;i<m;i++){
            int v=sc.nextInt();
            l1.add(v);
        }

        int n=sc.nextInt();
        LinkedList<Integer> l2=new LinkedList<>();
```

```

        for(int i=0;i<n;i++){
            int v=sc.nextInt();
            l2.add(v);
        }

        l1.addAll(l2);
        Collections.sort(l1);

        for(int i:l1)
            System.out.print(i+" ");
    }
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

Rahul is working on a list manipulation problem where he needs to reverse a specific subarray using a stack. Given an array and two indices l and r , he wants to reverse only the portion of the array from index l to r (both inclusive) while keeping the rest of the array unchanged.

Since Rahul wants to solve this problem efficiently, he decides to use a stack to reverse the subarray in $O(r - l)$ time.

Your task is to help Rahul by implementing this functionality.

Input Format

The first line contains an integer n , the size of the array.

The second line contains n space-separated integers $arr[i]$.

The third line contains two integers l and r , denoting the start and end indices of the subarray to reverse.

Note: The array follows 0-based indexing.

Output Format

The output prints the modified array after reversing the subarray between indices

l and r.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6

1 2 3 4 5 6

1 4

Output: 1 5 4 3 2 6

Answer

```
import java.util.*;

public class Main {
    // You are using Java

    public static void main(String[] args) {
        Scanner sc=new Scanner(System.in);

        int n=sc.nextInt();
        int[] arr=new int[n];

        for(int i=0;i<n;i++)
            arr[i]=sc.nextInt();

        int l=sc.nextInt();
        int r=sc.nextInt();

        Stack<Integer> stack=new Stack<>();

        for(int i=l;i<=r;i++)
            stack.push(arr[i]);

        for(int i=l;i<=r;i++)
            arr[i]=stack.pop();

        for(int i=0;i<n;i++)
            System.out.print(arr[i]+" ");
```

Status : Correct

Marks : 10/10

4. Problem Statement

Rahul, a stock trader, wants to analyze the stock prices of a company over several days. For each day, he wants to determine the stock span, which is the number of consecutive days (including the current day) where the stock price is less than or equal to the price on that day.

The stock span helps him understand how long a stock has been continuously increasing or staying the same. You need to help Rahul by computing the stock span for each day using a Stack data structure efficiently.

Example:

Input:

7

100 80 60 70 60 75 85

Output:

1 1 1 2 1 4 6

Explanation:

For each day:

Day 1: Price = 100 Span = 1 (Only this day)
Day 2: Price = 80 Span = 1 (Only this day)
Day 3: Price = 60 Span = 1 (Only this day)
Day 4: Price = 70 Span = 2 (Includes today and previous day)
Day 5: Price = 60 Span = 1 (Only this day)
Day 6: Price = 75 Span = 4 (Includes today and previous three days)
Day 7: Price = 85 Span = 6 (Includes today and previous five days)

Input Format

The first line contains an integer n, the number of days.

The second line contains n space-separated integers prices[i], where prices[i] represents the stock price on the i-th day.

Output Format

The output prints n space-separated integers representing the stock span for each day.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 7

100 80 60 70 60 75 85

Output: 1 1 1 2 1 4 6

Answer

// You are using Java

import java.io.*;

import java.util.*;

class Main{

public static void main(String[] args){

Scanner sc=new Scanner(System.in);

int n=sc.nextInt();

int[] arr=new int[n];

Stack<Integer> stack=new Stack<>();

for(int i=0;i<n;i++){

arr[i]=sc.nextInt();

int[] span=new int[n];

for(int i=0;i<n;i++){

while(!stack.isEmpty() && arr[stack.peek()]<=arr[i])

stack.pop();

if(stack.isEmpty())

span[i]=i+1;


```
        else
            span[i]=i-stack.peek();

        stack.push(i);
    }
    for(int i=0;i<n;i++)
        System.out.print(span[i]+" ");
    }
}
```

Status : Correct

Marks : 10/10